

ОТЧЁТ

по лабораторной работе № 4

**МАСШТАБИРОВАНИЕ ЧТЕНИЯ POSTGRESQL: PRIMARY +
REPLICA**

по модулю «Масштабирование баз данных»

Исполнитель:

обучающийся группы K0709-23/3

подпись, дата

У. И. Усачева

Руководитель:

преподаватель

подпись, дата

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Развёртывание Primary и Replica	7
1.1 Два экземпляра PostgreSQL	7
1.2 Кто есть кто и как подключиться	8
2 Настройка streaming replication.....	9
2.1 Как изменение попадает на Replica.....	9
2.2 Что пришлось настроить	9
2.3 Состояние репликации	10
3 Проверка того, что репликация работает	12
4 Режим только для чтения на Replica.....	13
5 Чтение сервиса через Replica.....	14
5.1 Как это устроено в коде.....	14
5.2 Доказательство: чей это был запрос	15
5.3 Состояние репликации глазами сервиса.....	16
6 Задержка репликации	17
6.1 Опыт 1. Без нагрузки	17
6.2 Опыт 2. Под нагрузкой	17
6.3 Опыт 3. Поймать устаревшее чтение.....	18
6.4 То же самое через API	18
7 Что именно масштабирует Read Scaling.....	20
8 Ответы на контрольные вопросы	22
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26
ПРИЛОЖЕНИЕ А Конфигурация Primary, Replica и сервиса	27

ПРИЛОЖЕНИЕ Б Инициализация Replica	30
ПРИЛОЖЕНИЕ В Чтение состояния репликации сервисом	31

ВВЕДЕНИЕ

Предыдущие работы улучшали один сервер базы данных: индексы ускоряли поиск, партиционирование уменьшало объём чтения и упрощало обслуживание больших таблиц. Но у одного сервера есть предел: когда читающих запросов становится больше, чем он способен обслужить, оптимизировать уже нечего — нужно добавлять серверы.

Цель работы — настроить для сервиса TaskManager схему PostgreSQL Primary + Replica со streaming replication, перевести часть чтения на Replica и понять, как изменения попадают на Replica и почему она может временно отставать от Primary.

Для достижения цели решались следующие задачи:

- поднять два экземпляра PostgreSQL и настроить между ними streaming replication;
- убедиться, что изменения с Primary доезжают до Replica, а запись на Replica невозможна;
- направить чтение списка задач и статистики сервиса на Replica, оставив запись на Primary;
- измерить задержку репликации без нагрузки, под нагрузкой и при остановленном применении WAL, а также поймать устаревшее чтение через API;
- измерить, что именно даёт Read Scaling: скорость запроса или пропускную способность.

Работа выполнена на сервисе TaskManager из предыдущих лабораторных работ: .NET 8, EF Core, PostgreSQL 16 в Docker. При выполнении использовались документация PostgreSQL [1–6], методические указания [7], статья Э. Брюера о CAP-теореме [8] и инструмент записи терминала VHS [9].

Таблица 1 – Среда выполнения экспериментов

Параметр	Значение
СУБД	PostgreSQL 16.14 (postgres:16-alpine), два экземпляра в

Параметр	Значение
	Docker Desktop, Windows 11
Ресурсы Docker	20 CPU, 10,7 ГБ памяти на оба контейнера вместе
Primary	контейнер taskmanager-postgres, порт 5432
Replica	контейнер taskmanager-postgres-replica, порт 5433
Репликация	streaming replication, асинхронная, через слот replica_slot
Сервис	TaskManager: .NET 8, EF Core 8, Dapper, Liquibase, Redis
Данные	1 000 000 задач, 796 872 связи задача-тег, 10 001 пользователь, база 677 МБ

Все эксперименты автоматизированы:

Таблица 2 – Скрипты и результаты

Часть	Запуск	Результаты
1–2 — роли экземпляров и состояние репликации	bash scripts/lab-04/01-replication-status.sh	docs/lab-04/results/01-status.txt
3 — изменения доезжают до Replica	bash scripts/lab-04/02-prove-replication.sh	02-prove.txt
4 — Replica только для чтения	bash scripts/lab-04/03-read-only.sh	03-read-only.txt
6 — replication lag	bash scripts/lab-04/04-lag.sh	04-lag.txt, 04-lag-samples.csv
5 — чтение сервиса идёт на Replica	bash scripts/lab-04/05-api-read-path.sh	05-api.txt
Что масштабируется	bash scripts/lab-04/06-read-scaling.sh, 07-cpu-limited-scaling.sh	06-read-scaling.txt, 07-cpu-limited.txt
Всё подряд	bash scripts/lab-04/run-all.sh	docs/lab-04/results/
Запись терминала	vhs scripts/lab-04/demo.tape	docs/lab-04/demo.gif

Как измерялось. Состояние репликации берётся из системных представлений PostgreSQL: `pg_stat_replication` на Primary и `pg_stat_wal_receiver` на Replica. Отставание считается двумя способами: в байтах WAL — `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)`, и во времени — поля `write_lag`, `flush_lag`, `replay_lag`. Замеры под нагрузкой складывает функция `lab04.sample_lag` (`scripts/lab-04/00-setup.sql`): она раз в 0,2 секунды пишет строку в таблицу `lab04.lag_samples`.

Запись терминала. GIF снят инструментом VHS на работающем сервисе: роли экземпляров, streaming replication, отказ Replica принимать запись и момент, когда Replica отстала от Primary.

```
> powershell -ExecutionPolicy Bypass -File scripts/lab-04/demo-replication.ps1 -Phase basics -Pause 2

=== Два экземпляра PostgreSQL: pg_is_in_recovery() - f y Primary, t y Replica
instance | in_recovery | lsn
-----+-----+-----
primary :5432 | f       | 0/AC171E48
(1 row)

instance | in_recovery | lsn
-----+-----+-----
replica :5433 | t       | 0/AC171E48
(1 row)

=== Streaming replication глазами Primary
application_name | state | sync_state | lag_bytes
-----+-----+-----+-----
taskmanager-replica | streaming | async | 0
(1 row)

=== Пишем на Primary
DELETE 0
INSERT 0 1

=== Читаем на Replica: строка уже здесь
name | color | created_at
-----+-----+-----
lab04-demo | #00ff00 | 2026-09-13 23:20:46.275527+00
(1 row)

=== Пробуем писать на Replica
ERROR: cannot execute INSERT in a read-only transaction
> █
```

Рисунок 1 – Primary и Replica: pg_is_in_recovery, состояние streaming, запись на Primary видна на Replica, попытка записи на Replica отклонена

```
=== Останавливаем применение WAL на Replica: pg_wal_replay_pause()
pg_wal_replay_pause | paused
-----+-----+-----
instance | name | color
-----+-----+-----
primary | lab04-demo | #ff0000
(1 row)

=== Меняем цвет тега на Primary
UPDATE 1

=== Primary отдаёт новое значение, Replica - старое: это replication lag
instance | name | color
-----+-----+-----
primary | lab04-demo | #ff0000
(1 row)

instance | name | color
-----+-----+-----
replica | lab04-demo | #00ff00
(1 row)

sent_lsn | replay_lsn | lag_bytes
-----+-----+-----
0/AC172068 | 0/AC171FA0 | 200
(1 row)

=== Сервис показывает то же самое: GET /health/replica
{"status":"Healthy","checks":[{"name":"replica","status":"Healthy","description":"The replica is streaming","data":{"replicaConfigured":true,"primaryLsn":"0/AC172068","replicaReplayLsn":"0/AC171FA0","lagBytes":200,"lagSeconds":27.504035,"connections":1}}]}

=== Возобновляем применение WAL
pg_wal_replay_resume
-----+-----+-----
instance | name | color
-----+-----+-----
replica | lab04-demo | #ff0000
(1 row)

DELETE 1
> █
```

Рисунок 2 – Применение WAL на Replica остановлено: Primary отдаёт новое значение, Replica — старое, отставание 200 байт WAL

1 Развёртывание Primary и Replica

1.1 Два экземпляра PostgreSQL

Второй экземпляр добавлен в `docker-compose.yml` рядом с первым. Primary — обычный контейнер `postgres:16-alpine` с явно заданными параметрами репликации:

Листинг 1 – Primary в `docker-compose.yml`

```
postgres:
  image: postgres:16-alpine
  container_name: taskmanager-postgres
  command:
    - postgres
    - -c
    - hba_file=/etc/postgresql/pg_hba.conf
    - -c
    - wal_level=replica
    - -c
    - max_wal_senders=10
    - -c
    - max_replication_slots=10
    - -c
    - max_slot_wal_keep_size=2GB
    - -c
    - cluster_name=taskmanager-primary
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ../docker/postgres/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
```

Листинг 2 – Replica в `docker-compose.yml`

```
postgres-replica:
  image: postgres:16-alpine
  container_name: taskmanager-postgres-replica
  depends_on:
    postgres:
      condition: service healthy
  entrypoint: ["/bin/sh", "/replica-entrypoint.sh"]
  command:
    - postgres
    - -c
    - hba_file=/etc/postgresql/pg_hba.conf
    - -c
    - hot_standby=on
    - -c
    - hot_standby_feedback=on
    - -c
    - cluster_name=taskmanager-replica
  ports:
    - "5433:5432"
  volumes:
    - postgres_replica_data:/var/lib/postgresql/data
    - ../docker/postgres/replica-entrypoint.sh:/replica-entrypoint.sh:ro
    - ../docker/postgres/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U taskmanager -d taskmanager_db && psql -U taskmanager -d taskmanager_db -tAc 'SELECT pg_is_in_recovery()' | grep -q t"]
```

Реплика поднимается сама: при первом запуске её `entrypoint` (`docker/postgres/replica-entrypoint.sh`, приложение Б) дожидается Primary, создаёт на нём роль `replicator` и слот репликации, снимает базовую копию через `pg_basebackup --write-recovery-conf` и запускает PostgreSQL в режиме `standby`. Весь стенд поднимается одной командой `docker compose up -d`.

1.2 Кто есть кто и как подключиться

Роль экземпляра определяет функция `pg_is_in_recovery()`: на Primary она возвращает `f`, на Replica — `t`, потому что Replica постоянно находится в процессе восстановления из WAL.

Листинг 3 – Роли экземпляров

instance	in_recovery	wal_lsn	cluster_name
primary (порт 5432)	f	0/527E48E8	taskmanager-primary
instance	in_recovery	replay_lsn	cluster_name
replica (порт 5433)	t	0/527E48E8	taskmanager-replica

Таблица 3 – Как подключиться к каждому экземпляру

Куда	Команда или строка подключения
Primary из контейнера	<code>docker compose exec postgres psql -U taskmanager -d taskmanager_db</code>
Replica из контейнера	<code>docker compose exec postgres-replica psql -U taskmanager -d taskmanager_db</code>
Primary с хоста	<code>psql -h localhost -p 5432 -U taskmanager -d taskmanager_db</code>
Replica с хоста	<code>psql -h localhost -p 5433 -U taskmanager -d taskmanager_db</code>
Сервис: запись	<code>ConnectionStrings__DefaultConnection = Host=postgres;Port=5432;...</code>
Сервис: чтение	<code>ConnectionStrings__ReplicaConnection = Host=postgres-replica;Port=5432;...</code>

2 Настройка streaming replication

2.1 Как изменение попадает на Replica

Цепочка одинакова для любой записи:

- 1) Primary меняет данные в своих страницах памяти.
- 2) Изменение записывается в WAL (Write-Ahead Log) — журнал предзаписи. Транзакция считается зафиксированной, когда её запись в WAL сброшена на диск.
- 3) Процесс `walsender` на Primary отправляет поток WAL реплике по обычному подключению к порту 5432, но к «базе» `replication`.
- 4) Процесс `walreceiver` на Replica принимает поток, записывает его у себя и сбрасывает на диск.
- 5) Процесс восстановления (`startup`) применяет записи WAL к файлам данных Replica. После этого изменение видно запросам на Replica.

Асинхронность означает, что Primary не ждёт шагов 3–5: он отвечает клиенту «готово» сразу после шага 2. Отсюда и берётся `replication lag` (часть 6).

2.2 Что пришлось настроить

Таблица 4 – Параметры репликации на Primary

Параметр	Значение	Зачем
<code>wal_level</code>	<code>replica</code>	В WAL пишется достаточно данных, чтобы другой сервер мог их применить
<code>max_wal_senders</code>	10	Сколько реплик могут читать поток WAL одновременно
<code>max_replication_slots</code>	10	Сколько слотов репликации можно создать
<code>max_slot_wal_keep_size</code>	2 ГБ	Ограничение на WAL, который Primary хранит ради отставшей Replica
<code>hot_standby</code> (на Replica)	<code>on</code>	Разрешает читать с Replica, пока она применяет WAL
<code>hot_standby_feedback</code> (на Replica)	<code>on</code>	Replica сообщает Primary, какие версии строк ей ещё нужны

Отдельная настройка — файл `pg_hba.conf`. Стандартный файл разрешает подключения к «базе» replication только с localhost, поэтому Replica из соседнего контейнера получала бы отказ. В проект добавлен общий `docker/postgres/pg_hba.conf`, он подключается обоим экземплярам параметром `hba_file`:

Листинг 4 – Правила подключения для репликации

```
# Приложение и psql с хоста: только по паролю
host      all             all             all             scram-sha-256

# Подключения репликации: Replica приходит по сети docker как роль replicator
local     replication     all             trust
host      replication     all             all             scram-sha-256
```

Слот репликации. Replica подключена через слот `replica_slot`. Слот — это отметка на Primary о том, какие WAL уже получила Replica. Пока слот существует, Primary не удаляет неотданные WAL, даже если Replica выключена, и после включения она догонит Primary без повторного `pg_basebackup`. Обратная сторона: если Replica выключена надолго, WAL копятся и занимают диск, поэтому задан предел `max_slot_wal_keep_size = 2GB`.

2.3 Состояние репликации

Листинг 5 – `pg_stat_replication` на Primary

```
application_name | taskmanager-replica
client_addr      | 172.20.0.4
state            | streaming
sync_state       | async
sent_lsn         | 0/527E48E8
write_lsn        | 0/527E48E8
flush_lsn        | 0/527E48E8
replay_lsn       | 0/527E48E8
lag_bytes        | 0
```

`state = streaming` означает, что Replica догнала Primary и получает изменения потоком, `sync_state = async` — что Primary не ждёт подтверждений от Replica. Четыре позиции LSN показывают, до какого места WAL отправлен, записан, сброшен на диск и применён.

Листинг 6 – `pg_stat_wal_receiver` на Replica

```
status           | streaming
```

sender_host		postgres
sender_port		5432
slot_name		replica_slot
written_lsn		0/527E48E8
latest_end_lsn		0/527E48E8
last_msg_receipt_time		2026-09-13 23:02:42.361368+00
conninfo		user=replicator password=***** dbname=replication host=postgres
port=5432 ...		

3 Проверка того, что репликация работает

Скрипт `02-prove-replication.sh` выполняет запись на Primary и сразу читает ту же строку на Replica.

Листинг 7 – Запись на Primary и чтение на Replica

```
=== 1. Запись на Primary: INSERT в tasks (id = 58089a12-2195-47ea-a1f1-09ee1c813e3b)
id          | 58089a12-2195-47ea-a1f1-09ee1c813e3b
title       | ЛР4: проверка репликации 2026-09-13 23:05:47 UTC
created_at  | 2026-09-13 23:05:48.511966+00
primary_wal_lsn | 0/52822900

=== 2. Чтение на Replica: та же строка
id          | 58089a12-2195-47ea-a1f1-09ee1c813e3b
title       | ЛР4: проверка репликации 2026-09-13 23:05:47 UTC
created_at  | 2026-09-13 23:05:48.511966+00
replica_replay_lsn | 0/52822900
last_xact_replayed | 2026-09-13 23:05:48.517088+00
```

Позиции WAL на обоих экземплярах совпали, строка на Replica та же самая. Далее скрипт проверяет остальные операции:

Таблица 5 – Все виды изменений доезжают до Replica

Операция на Primary	Что видит Replica
INSERT задачи	строка появилась
UPDATE: изменены title и status	новые значения
DELETE задачи	строк 0

После всех операций `pg_stat_replication` показывает `state = streaming` и `lag_bytes = 0`: Replica применила весь WAL.

4 Режим только для чтения на Replica

Скрипт 03-read-only.sh пробует выполнить на Replica запись всех видов. PostgreSQL отклоняет каждую попытку:

Таблица 6 – Попытки записи на Replica

Запрос на Replica	Ответ PostgreSQL	SQLSTATE
SELECT count(*) FROM tasks	1 000 000 — чтение работает	—
INSERT INTO tags ...	cannot execute INSERT in a read-only transaction	25006
UPDATE tasks SET ...	cannot execute UPDATE in a read-only transaction	25006
DELETE FROM tags ...	cannot execute DELETE in a read-only transaction	25006
CREATE TABLE replica_test ...	cannot execute CREATE TABLE in a read-only transaction	25006
BEGIN READ WRITE	cannot set transaction read-write mode during recovery	0A000

Сам экземпляр сообщает о своём режиме так: `pg_is_in_recovery() = t, transaction_read_only = on, hot_standby = on.`

Почему приложение не должно использовать Replica как отдельную базу для записи. Replica не принимает запись не из-за настройки, которую можно было бы отключить, а по устройству: она непрерывно применяет чужой WAL. Если бы на ней разрешили менять данные, её файлы разошлись бы с потоком WAL от Primary, и применить следующую запись журнала стало бы невозможно. Кроме того, у Primary и Replica общая история: две независимо принимающие запись копии — это уже два разных кластера, между которыми пришлось бы решать конфликты. Поэтому вся запись идёт на Primary, а Replica используется только для чтения.

5 Чтение сервиса через Replica

5.1 Как это устроено в коде

В сервисе появилось второе подключение к базе. Модель EF Core та же самая, отличается только строка подключения, поэтому запросы не пришлось писать заново:

Таблица 7 – Компоненты чтения с Replica

Компонент	Слой	Роль
ReplicaDbContext : AppDbContext	Infrastructure	Тот же набор сущностей, но на строке подключения к Replica и без отслеживания изменений (NoTracking)
ITaskReadRepository	Application	Интерфейс только с читающими методами: записать через него нельзя
TaskReadRepository : TaskRepository	Infrastructure	Те же запросы, что и у обычного репозитория, но на ReplicaDbContext
TaskService	Application	Решает, какой запрос куда отправить
IReplicationMonitor	Application, Infrastructure	Читает pg_stat_replication и состояние Replica для /api/replication и /health/replica

Листинг 8 – Регистрация двух подключений в Program.cs

```
// Configure Database: writes and read-your-writes go to the primary
var primaryConnection = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseNpgsql(primaryConnection);
});

// Read replica: lists and statistics are served from here. Without a replica connection
// the same context points at the primary, so the service works with a single server too.
var replicaConnection = builder.Configuration.GetConnectionString("ReplicaConnection");
builder.Services.AddDbContext<ReplicaDbContext>(options =>
{
    options.UseNpgsql(string.IsNullOrEmpty(replicaConnection) ? primaryConnection :
replicaConnection);
    options.UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking);
});
```

Если строка подключения к Replica не задана, оба контекста смотрят на Primary, и сервис работает на одном сервере — это удобно для локальной отладки.

Таблица 8 – Маршрутизация запросов сервиса

Операция API	Куда идёт	Почему
GET /api/tasks — список задач	Replica	Основной читающий сценарий, список допускает отставание на доли секунды
GET /api/tasks/stats — статистика по статусам	Replica	Тяжёлое чтение по всей таблице
POST, PUT, DELETE /api/tasks	Primary	Любая запись возможна только на Primary
Чтение задачи сразу после создания или изменения	Primary	Read-your-writes: на Replica изменения может ещё не быть
GET /api/tasks/{id}	Primary	Клиент часто открывает задачу сразу после её создания
Проверка прав (IsOwnerAsync)	Primary	Решение о доступе не должно приниматься по устаревшим данным

Выбор зафиксирован не только в коде, но и в тестах: `GetTasksAsync_ShouldNotTouchThePrimary` проверяет, что список берётся только с Replica, а `CreateAsync_ShouldWriteToPrimaryAndReadItBackFromPrimary` — что запись и чтение сразу после неё идут на Primary.

5.2 Доказательство: чей это был запрос

Скрипт `05-api-read-path.sh` включает на обоих экземплярах логирование всех запросов (`log_min_duration_statement = 0`), очищает кэш Redis, вызывает два endpoint'а и смотрит, в чей журнал попал запрос.

Листинг 9 – SELECT списка задач в журнале Replica

```
2026-09-13 23:10:54.172 UTC [777] LOG: duration: 0.265 ms execute <unnamed>:
SELECT t0.id, t0.created_at, t0.description, t0.due_date, t0.priority,
t0.project_id,
       t0.status, t0.title, t0.updated_at, t0.user_id, p.id, u.id, t1.task_id, ...
FROM (
  SELECT t.id, t.created_at, ... FROM tasks AS t
  ORDER BY t.created_at DESC
  LIMIT $1 OFFSET $2
) AS t0 ...
```

Листинг 10 – INSERT новой задачи в журнале Primary

```
2026-09-13 23:10:54.231 UTC [208] LOG: duration: 0.399 ms execute <unnamed>:
INSERT INTO tasks (id, created_at, description, due_date, priority, project_id, status, title,
updated_at, user_id)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
DETAIL: parameters: $1 = '2ddaf2e7-3bc0-4ccf-964a-a08c82f9bb43', ...,
$8 = 'LR4: task created through API', ...
```

Таблица 9 – Где какой запрос оказался

Запрос	Журнал Primary	Журнал Replica
GET /api/tasks?pageSize=5 (список)	нет	6 запросов к tasks
POST /api/tasks (создание)	INSERT INTO tasks	0

5.3 Состояние репликации глазами сервиса

Сервис сам показывает, что происходит с репликацией: GET /api/replication (роль Admin) и GET /health/replica.

Листинг 11 – Ответ GET /health/replica

```
{
  "status": "Healthy",
  "checks": [
    {
      "name": "replica",
      "status": "Healthy",
      "description": "The replica is streaming",
      "data": {
        "replicaConfigured": true,
        "primaryLsn": "0/70F3B270",
        "replicaReplayLsn": "0/70F3B270",
        "lagBytes": 0,
        "lagSeconds": 11.697,
        "connections": 1
      }
    }
  ]
}
```

Проверка считает Replica отставшей, только если она отстала одновременно по объёму WAL (больше 16 МБ) и по времени (больше 30 секунд). Одного времени недостаточно: pg_last_xact_replay_timestamp() возвращает время последней применённой транзакции, и в тишине, когда на Primary никто не пишет, это значение стареет само по себе, хотя Replica полностью догнала Primary.

6 Задержка репликации

6.1 Опыт 1. Без нагрузки

Скрипт `04-lag.sh` заранее запускает на Replica ожидание строки, затем вставляет её на Primary с `created_at = clock_timestamp()`. Когда строка появляется на Replica, та сравнивает своё текущее время с временем создания строки.

Таблица 10 – Задержка появления строки на Replica, пять замеров подряд

Замер	1	2	3	4	5
Задержка, мс	144,1	271,8	118,8	175,5	142,6

В эти 120–270 мс входит не только пересылка WAL: сюда попадает фиксация транзакции на самом Primary и запись WAL на диск на Replica. Обе операции в Docker Desktop идут через виртуализированную файловую систему и стоят десятки миллисекунд. На сервере с локальными SSD и сетью внутри одного датацентра такая задержка обычно составляет единицы миллисекунд. Важно другое: нулём она не бывает.

6.2 Опыт 2. Под нагрузкой

На Primary выполняется UPDATE задач за последние 6 месяцев (около 240 тысяч строк), параллельно раз в 0,2 секунды снимается отставание.

Таблица 11 – Отставание Replica во время массового UPDATE

Время от начала, мс	Отставание, байт	Отставание	replay_lag, мс
815	5 308 448	5,1 МБ	279,3
2 839	7 716 904	7,4 МБ	248,6
5 001	4 169 760	4,0 МБ	309,6
7 136	4 595 776	4,4 МБ	301,5
8 182	0	0	47,6
9 191	96	96 байт	1,5

Максимум за прогон — 30,5 МБ неприменённого WAL. Пока идёт запись, Replica отстаёт на мегабайты журнала и примерно на 0,3 секунды; через секунду после конца нагрузки отставание снова нулевое.

Отдельно стоит отметить поведение полей `write_lag`, `flush_lag` и `replay_lag`: между записями они не обнуляются, а сохраняют последнее измеренное значение. Поэтому в конце прогона в них остаётся большое число (в одном из прогонов — 7 секунд), хотя `lag_bytes` уже равен нулю. Для мониторинга живого отставания надёжнее сравнивать позиции LSN, что и делает `health check` сервиса.

6.3 Опыт 3. Поймать устаревшее чтение

Чтобы увидеть момент, когда Primary уже содержит новое значение, а Replica ещё нет, применение WAL на Replica останавливается функцией `pg_wal_replay_pause()`. Приём WAL при этом продолжается: реплика получает журнал, но не применяет его.

Листинг 12 – Остановленное применение WAL: Primary и Replica расходятся

```
=== Останавливаем применение WAL на Replica: pg_wal_replay_pause()
pg_wal_replay_pause | paused
-----+-----
| t

=== Меняем цвет тега на Primary
UPDATE 1

=== Primary отдаёт новое значение, Replica — старое: это replication lag
instance | name | color
-----+-----
primary | lab04-demo | #ff0000

instance | name | color
-----+-----
replica | lab04-demo | #00ff00

sent_lsn | replay_lsn | lag_bytes
-----+-----
0/AC172068 | 0/AC171FA0 | 200

=== Возобновляем применение WAL
instance | name | color
-----+-----
replica | lab04-demo | #ff0000
```

6.4 То же самое через API

Тот же опыт на уровне сервиса: при остановленном применении WAL создаётся задача через `POST /api/tasks` (Primary принимает её), после чего

запрашивается список GET /api/tasks?pageSize=1&sort=-created_at (идёт на Replica).

Листинг 13 – Устаревший список задач через API

```
--- POST /api/tasks (Primary принимает запись)
HTTP 201
--- GET /api/tasks?pageSize=1&sort=-created_at — первая задача списка по версии
Replica:
"title":"LR4: task created through API"
--- та же задача на Primary:
LR4: created while replay is paused
--- возобновляем применение WAL и повторяем GET
"title":"LR4: created while replay is paused"
```

Клиент получил ответ «задача создана», но в списке её ещё нет. Это не ошибка сервиса, а следствие асинхронной репликации. Поэтому в TaskManager чтение задачи сразу после создания идёт на Primary, а через Replica читаются только списки и статистика, для которых отставание в доли секунды несущественно.

Главный вывод. Репликация не означает мгновенную синхронизацию. Между фиксацией транзакции на Primary и её применением на Replica всегда есть задержка: в покое это сотни миллисекунд в нашей среде, под нагрузкой — десятки мегабайт неприменённого WAL, а при остановке применения она может быть сколь угодно большой.

7 Что именно масштабирует Read Scaling

Чтобы ответить на контрольный вопрос 7 измерением, одна и та же читающая нагрузка запускалась в двух вариантах: целиком на Primary и поровну между Primary и Replica. Нагрузку создаёт `pgbench` запросом «сколько задач создано в случайном месяце» — он повторяет форму запросов сервиса и читает партицию по `created_at`.

Таблица 12 – Читающая нагрузка на общих ядрах хоста (20 CPU на оба контейнера)

Вариант	Клиентов	Запросов в секунду	Средняя задержка, мс
A. Только Primary	16	1460,5	11,0
B. Primary	8	720,9	11,1
B. Replica	8	716,0	11,2
B. Суммарно	16	1436,9 (−1,6 %)	—
C. Primary, пока на нём идёт постоянный UPDATE	8	679,1	11,8
C. Replica, пока Primary занят тем же UPDATE	8	637,0 (−6,2 %)	12,6

Прироста нет, и это ожидаемо: оба контейнера делят одни и те же ядра и один диск. Выносить чтение на реплику, которая живёт на том же железе, бессмысленно — суммарная мощность не изменилась. Вариант C показывает и то, что реплика не бесплатна: она сама тратит ресурсы на применение WAL, поэтому под потоком изменений читает даже чуть медленнее Primary.

Чтобы увидеть эффект Read Scaling в чистом виде, каждый экземпляр ограничен четырьмя ядрами — так ведут себя два отдельных сервера (наложение `docker/lab-04-cpu-limit.yml`, скрипт `07-cpu-limited-scaling.sh`):

Таблица 13 – Та же нагрузка, когда у каждого сервера свои 4 ядра

Вариант	Клиентов	Запросов в секунду	Средняя задержка, мс
A. Только Primary (4 ядра)	16	271,2	59,0
B. Primary (4 ядра)	8	432,6	18,5
B. Replica (4 ядра)	8	436,0	18,3
B. Суммарно	16	868,6 (+220 %)	—

Шестнадцать клиентов на одном четырёхъядерном сервере конкурируют за процессор и дают 271 запрос в секунду при средней задержке 59 мс. Те же шестнадцать клиентов, разложенные по двум серверам, дают 869 запросов в секунду при задержке 18 мс. Сам запрос быстрее не стал: на общих ядрах хоста, где очереди нет, задержка одинаковая в обоих вариантах — около 11 мс. Уменьшилась очередь, а не время выполнения SQL.

Вывод. Read Scaling увеличивает не скорость отдельного запроса, а количество чтений, которое система способна обслужить, и только при условии, что у реплики свои вычислительные ресурсы.

8 Ответы на контрольные вопросы

1) **Чем Primary отличается от Replica?** Primary — единственный экземпляр, который принимает изменения: он выполняет INSERT, UPDATE, DELETE, CREATE TABLE и пишет их в свой WAL. Replica — копия Primary, которая непрерывно применяет его WAL и находится в режиме восстановления (`pg_is_in_recovery() = t`). Она обслуживает только чтение: любая попытка записи отклоняется с ошибкой 25006.

2) **Почему запись выполняем на Primary?** Потому что источник истины и порядок изменений должен быть один. Replica физически повторяет содержимое Primary, применяя его журнал; если бы она принимала собственную запись, её файлы разошлись бы с потоком WAL и применить следующую запись журнала стало бы невозможно. Две независимо принимающие запись копии — это уже два разных кластера с конфликтами, которые кто-то должен разрешать.

3) **Как изменение из Primary попадает на Replica?** Изменение фиксируется в WAL на Primary. Процесс `walsender` передаёт поток WAL по сети процессу `walreceiver` на Replica. Replica записывает полученный WAL у себя и сбрасывает на диск, а процесс восстановления применяет записи к файлам данных. После применения изменение видно запросам на Replica. Позиции каждого этапа видны в `pg_stat_replication`: `sent_lsn`, `write_lsn`, `flush_lsn`, `replay_lsn`.

4) **Что такое WAL в контексте репликации?** WAL (Write-Ahead Log) — журнал предзаписи: последовательность записей обо всех изменениях страниц данных. PostgreSQL сначала пишет изменение в WAL и только потом меняет сами файлы данных, поэтому WAL достаточно для восстановления после сбоя. Репликация использует тот же журнал как канал передачи изменений: Replica не выполняет SQL-запросы Primary заново, а применяет готовые физические изменения страниц из WAL.

5) **Что такое replication lag?** Это отставание Replica от Primary: разница между тем, что Primary уже зафиксировал, и тем, что Replica уже применила. Измеряется в байтах WAL (`pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)`) или во времени (`replay_lag, now() - pg_last_xact_replay_timestamp()`). В измерениях этой работы отставание без нагрузки составляло сотни миллисекунд, под массовым UPDATE доходило до 30,5 МБ неприменённого WAL, а при остановленном применении росло неограниченно.

6) **Почему следующий SELECT после INSERT может увидеть старые данные, если отправить его на Replica?** Репликация асинхронная: Primary отвечает клиенту «зафиксировано», как только записал WAL себе на диск, и не ждёт, пока Replica получит и применит эту запись. Между этими моментами проходит время, и запрос, пришедший на Replica в этом промежутке, честно вернёт предыдущее состояние. В работе это воспроизведено дважды: на уровне SQL (Primary отдаёт `#ff0000`, Replica — `#00ff00`) и на уровне API (задача создана, но её ещё нет в списке). Поэтому сценарии read-your-writes направляются на Primary.

7) **Что именно масштабируется при Read Scaling: скорость одного SQL-запроса или способность обслуживать больше чтений?** Способность обслуживать больше чтений. Один и тот же запрос на Replica выполняется столько же времени, сколько на Primary: план и объём данных те же, средняя задержка в измерении была одинаковой — около 11 мс. Растёт суммарная пропускная способность, и только вместе с ресурсами: при ограничении в 4 ядра на сервер разделение нагрузки подняло её с 271 до 869 запросов в секунду, а на общих ядрах хоста прироста не было вовсе.

8) **Почему наличие Replica не отменяет необходимость индексов и оптимизации SQL?** Replica выполняет те же самые запросы по тем же данным и с теми же планами: запрос без индекса, читающий миллион строк, останется таким же медленным, просто эту работу будет делать другой сервер. Кроме того, реплика применяет весь WAL, который порождает

Primary, поэтому неоптимальная запись нагружает и её. Реплика добавляет мощности для параллельных чтений, но не уменьшает стоимость отдельного запроса: индексы, партиционирование и разумный SQL из предыдущих работ остаются обязательными.

9) Расскажите про CAP-теорему. CAP-теорема утверждает, что распределённая система не может одновременно гарантировать три свойства: Consistency (любое чтение видит последнюю подтверждённую запись), Availability (каждый запрос получает ответ) и Partition tolerance (система продолжает работать при потере связи между узлами). Сеть рвётся независимо от нашего желания, поэтому реальный выбор в момент разрыва идёт между C и A. Схема из этой работы — Primary с асинхронной Replica — это выбор в пользу доступности: если связь между узлами пропадёт, Replica продолжит отвечать, но будет отдавать устаревшие данные, то есть согласованность здесь в конечном счёте (eventual consistency). Настройка `synchronous_commit = remote_apply` смещает выбор в сторону согласованности: Primary ждёт подтверждения от Replica, и при её недоступности запись останавливается — страдает доступность. Расширение PACELC добавляет вторую половину картины: даже без разрывов (Else) приходится выбирать между Latency и Consistency, потому что синхронная репликация стоит задержки на каждом коммите. В TaskManager выбрана асинхронная репликация, а риск устаревшего чтения закрыт маршрутизацией: то, что требует немедленной согласованности, читается с Primary.

ЗАКЛЮЧЕНИЕ

В работе развёрнута схема PostgreSQL Primary + Replica и настроена асинхронная streaming replication через слот репликации. Реплика поднимается автоматически: при первом запуске `docker compose up -d` её entrypoint снимает базовую копию Primary через `pg_basebackup` и стартует в режиме standby, после чего `pg_stat_replication` на Primary показывает состояние streaming.

Проверено, что репликация работает: INSERT, UPDATE и DELETE на Primary появляются на Replica, позиции LSN совпадают. Проверено и обратное: любая запись на Replica отклоняется с ошибкой 25006, а явная транзакция на запись не открывается вовсе.

В сервисе TaskManager чтение списка задач и статистики переведено на Replica через отдельный контекст EF Core и репозиторий только для чтения; запись и чтение сразу после записи остались на Primary. Маршрутизация доказана журналами обоих серверов: SELECT списка задач попал в журнал Replica, INSERT новой задачи — в журнал Primary.

Измерена задержка репликации: без нагрузки 120–270 мс в этой среде, под массовым UPDATE — до 30,5 МБ неприменённого WAL. При остановленном применении WAL удалось поймать момент, когда Primary уже отдаёт новое значение, а Replica и построенный на ней список задач — старое. Дополнительный эксперимент показал, что Read Scaling увеличивает пропускную способность чтения (271 → 869 запросов в секунду при разделении нагрузки между двумя четырёхъядерными серверами), но не ускоряет отдельный запрос и не даёт ничего, если реплика делит железо с Primary.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 PostgreSQL 16 Documentation. 27.2. Log-Shipping Standby Servers [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/warm-standby.html> (дата обращения: 14.09.2026).

2 PostgreSQL 16 Documentation. 30.1. Reliability and the Write-Ahead Log [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/wal-intro.html> (дата обращения: 14.09.2026).

3 PostgreSQL 16 Documentation. 28.2. The Cumulative Statistics System (pg_stat_replication) [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/monitoring-stats.html> (дата обращения: 14.09.2026).

4 PostgreSQL 16 Documentation. pg_basebackup [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/app-pgbasebackup.html> (дата обращения: 14.09.2026).

5 PostgreSQL 16 Documentation. 21.1. The pg_hba.conf File [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/auth-pg-hba-conf.html> (дата обращения: 14.09.2026).

6 PostgreSQL 16 Documentation. pgbench [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/pgbench.html> (дата обращения: 14.09.2026).

7 Лабораторная работа № 4. Масштабирование чтения PostgreSQL: Primary + Replica : методические указания. – Сириус : Колледж Университета «Сириус», 2026. – 3 с.

8 Brewer E. CAP twelve years later: How the "Rules" have changed // Computer. – 2012. – Vol. 45, № 2. – P. 23–29.

9 VHS — инструмент записи терминала [Электронный ресурс]. – URL: <https://github.com/charmbracelet/vhs> (дата обращения: 14.09.2026).

ПРИЛОЖЕНИЕ А

Конфигурация Primary, Replica и сервиса

Файл `docker-compose.yml` целиком: два экземпляра PostgreSQL, параметры репликации, строки подключения сервиса к Primary и Replica (разделы 1 и 5).

Листинг А.1 – `docker-compose.yml`

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: taskmanager-postgres
    # 64 MB /dev/shm по умолчанию не хватает параллельным VACUUM и запросам на миллионах строк
    shm_size: 512mb
    # ЛР №4: Primary — принимает запись и отдаёт WAL реплике. Параметры репликации заданы явно,
    # хотя в PostgreSQL 16 они и так по умолчанию такие: так их видно в конфигурации проекта
    command:
      - postgres
      - -c
      - hba_file=/etc/postgresql/pg_hba.conf
      - -c
      - wal_level=replica
      - -c
      - max_wal_senders=10
      - -c
      - max_replication_slots=10
      # Слот репликации держит WAL, пока Replica их не получит; ограничение не даёт
      # заполнить диск, если Replica надолго выключена
      - -c
      - max_slot_wal_keep_size=2GB
      # Имя экземпляра видно в логах и в pg_stat_replication
      - -c
      - cluster_name=taskmanager-primary
    environment:
      POSTGRES_USER: taskmanager
      POSTGRES_PASSWORD: taskmanager_password
      POSTGRES_DB: taskmanager_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./scripts:/scripts:ro
      - ./docker/postgres/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
    networks:
      - taskmanager-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskmanager -d taskmanager_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres-replica:
    image: postgres:16-alpine
    container_name: taskmanager-postgres-replica
    shm_size: 256mb
    depends_on:
      postgres:
        condition: service_healthy
    # При первом запуске entrypoint снимает базовую копию Primary через pg_basebackup
    entrypoint: ["/bin/sh", "/replica-entrypoint.sh"]
    command:
      - postgres
      - -c
      - hba_file=/etc/postgresql/pg_hba.conf
      # hot_standby разрешает читать с Replica, пока она применяет WAL
      - -c
      - hot_standby=on
      # Replica сообщает Primary, какие строки ей ещё нужны: длинные запросы на Replica
      # не будут отменяться из-за VACUUM на Primary
```

```

- -c
- hot_standby_feedback=on
# Имя реплики видно на Primary в pg_stat_replication.application_name
- -c
- cluster_name=taskmanager-replica
environment:
  POSTGRES_USER: taskmanager
  POSTGRES_PASSWORD: taskmanager_password
  POSTGRES_DB: taskmanager_db
  PRIMARY_HOST: postgres
  PRIMARY_PORT: 5432
  REPLICATION_USER: replicator
  REPLICATION_PASSWORD: replicator_password
  REPLICATION_SLOT: replica_slot
ports:
  - "5433:5432"
volumes:
  - postgres_replica_data:/var/lib/postgresql/data
  - ./docker/postgres/replica-entrypoint.sh:/replica-entrypoint.sh:ro
  - ./docker/postgres/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro
  - ./scripts:/scripts:ro
networks:
  - taskmanager-network
healthcheck:
  # Реплика здорова, когда принимает подключения и находится в режиме восстановления
  test: ["CMD-SHELL", "pg_isready -U taskmanager -d taskmanager_db && psql -U taskmanager -d taskmanager_db -tAc 'SELECT pg_is_in_recovery()' | grep -q t"]
  interval: 10s
  timeout: 5s
  retries: 10

redis:
  image: redis:7-alpine
  container_name: taskmanager-redis
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  networks:
    - taskmanager-network
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

liquibase:
  image: liquibase/liquibase:4.25
  container_name: taskmanager-liquibase
  depends_on:
    postgres:
      condition: service_healthy
  volumes:
    - ./docker/liquibase:/liquibase/changelog
  command: >
    --url=jdbc:postgresql://postgres:5432/taskmanager_db
    --username=taskmanager
    --password=taskmanager_password
    --changeLogFile=changelog/db.changelog-master.xml
  update
  networks:
    - taskmanager-network

api:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: taskmanager-api
  depends_on:
    postgres:
      condition: service_healthy
    postgres-replica:
      condition: service_healthy
    redis:
      condition: service_healthy
    liquibase:
      condition: service_completed_successfully

```

```

environment:
  ASPNETCORE_ENVIRONMENT: Development
  ConnectionStrings__DefaultConnection:
Host=postgres;Port=5432;Database=taskmanager_db;Username=taskmanager;Password=taskmanager_password
d
  # JP №4: чтение списков задач идёт на Replica, запись — на Primary
  ConnectionStrings__ReplicaConnection: Host=postgres-
replica;Port=5432;Database=taskmanager_db;Username=taskmanager;Password=taskmanager_password
  ConnectionStrings__Redis: redis:6379
  # Алерты о партициях (JP №3): токен и чат берутся из локального .env, он не коммитится
  Alerts__Telegram__BotToken: ${TELEGRAM_BOT_TOKEN:-}
  Alerts__Telegram__ChatId: ${TELEGRAM_CHAT_ID:-}
ports:
  - "5000:8080"
networks:
  - taskmanager-network

prometheus:
  image: prom/prometheus:v2.48.0
  container_name: taskmanager-prometheus
  ports:
    - "9090:9090"
  volumes:
    - ./docker/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--web.console.libraries=/usr/share/prometheus/console_libraries'
    - '--web.console.templates=/usr/share/prometheus/conssoles'
  networks:
    - taskmanager-network

grafana:
  image: grafana/grafana:10.2.2
  container_name: taskmanager-grafana
  ports:
    - "3000:3000"
  environment:
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: admin
    GF_USERS_ALLOW_SIGN_UP: false
  volumes:
    - grafana_data:/var/lib/grafana
    - ./docker/grafana/provisioning:/etc/grafana/provisioning
    - ./docker/grafana/dashboards:/var/lib/grafana/dashboards
  depends_on:
    - prometheus
  networks:
    - taskmanager-network

volumes:
  postgres_data:
  postgres_replica_data:
  redis_data:
  prometheus_data:
  grafana_data:

networks:
  taskmanager-network:
    driver: bridge

```

ПРИЛОЖЕНИЕ Б

Инициализация Replica

Entrypoint контейнера Replica: дожидается Primary, создаёт роль репликации и слот, снимает базовую копию через pg_basebackup и запускает PostgreSQL в режиме standby (подраздел 1,1).

Листинг Б.1 – replica-entrypoint.sh

```
#!/bin/sh
# ЛР №4: запуск Replica. Если каталог данных пуст, снимаем базовую копию Primary
# (pg_basebackup) и запускаем PostgreSQL в режиме standby: дальше он догоняет Primary
# по потоку WAL. Если данные уже есть, копия не снимается – сервер просто стартует.
set -e

PGDATA=${PGDATA:-/var/lib/postgresql/data}
PRIMARY_HOST=${PRIMARY_HOST:-postgres}
PRIMARY_PORT=${PRIMARY_PORT:-5432}
SLOT=${REPLICATION_SLOT:-replica_slot}

if [ ! -s "$PGDATA/Pg_VERSION" ]; then
    echo "Replica: жду Primary $PRIMARY_HOST:$PRIMARY_PORT"
    until PGPASSWORD="$POSTGRES_PASSWORD" psql -h "$PRIMARY_HOST" -p "$PRIMARY_PORT" \
        -U "$POSTGRES_USER" -d "$POSTGRES_DB" -tAc 'SELECT 1' > /dev/null 2>&1; do
        sleep 2
    done

    echo "Replica: создаю на Primary роль $REPLICATION_USER и слот $SLOT, если их ещё нет"
    PGPASSWORD="$POSTGRES_PASSWORD" psql -h "$PRIMARY_HOST" -p "$PRIMARY_PORT" \
        -U "$POSTGRES_USER" -d "$POSTGRES_DB" -v ON_ERROR_STOP=1 <<SQL
DO \\\$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = '$REPLICATION_USER') THEN
        CREATE ROLE $REPLICATION_USER WITH REPLICATION LOGIN PASSWORD '$REPLICATION_PASSWORD';
    END IF;
    -- Слот гарантирует, что Primary не удалит WAL, которые Replica ещё не получила
    IF NOT EXISTS (SELECT 1 FROM pg_replication_slots WHERE slot_name = '$SLOT') THEN
        PERFORM pg_create_physical_replication_slot('$SLOT');
    END IF;
END
\\\$;
SQL

    echo "Replica: снимаю базовую копию Primary"
    mkdir -p "$PGDATA"
    chown postgres:postgres "$PGDATA"
    chmod 0700 "$PGDATA"
    # -R создаёт standby.signal и primary_conninfo, -Xs тянет WAL потоком во время копирования,
    # -S подключает слот репликации, -c fast не ждёт очередной контрольной точки
    su-exec postgres env PGPASSWORD="$REPLICATION_PASSWORD" pg_basebackup \
        --host="$PRIMARY_HOST" --port="$PRIMARY_PORT" --username="$REPLICATION_USER" \
        --pgdata="$PGDATA" --format=plain --wal-method=stream --write-recovery-conf \
        --slot="$SLOT" --checkpoint=fast --progress
    echo "Replica: копия снята, запускаю PostgreSQL"
fi

exec docker-entrypoint.sh "$@"
```

ПРИЛОЖЕНИЕ В

Чтение состояния репликации сервисом

Класс `ReplicationRepository`: запросы к `pg_stat_replication` на Primary и к функциям восстановления на Replica, на которых построены `GET /api/replication` и `GET /health/replica` (подраздел 5,3).

Листинг В.1 – `ReplicationRepository.cs`

```
using Dapper;
using Microsoft.Extensions.Logging;
using Npgsql;
using TaskManager.Application.Interfaces;
using TaskManager.Application.Replication;

namespace TaskManager.Infrastructure.Repositories;

/// <summary>
/// Reads pg_stat_replication on the primary and the recovery state on the replica (Dapper).
/// </summary>
public class ReplicationRepository : IReplicationMonitor
{
    private const string PrimarySql = @"
        SELECT pg_is_in_recovery() AS InRecovery, pg_current_wal_lsn()::text AS Lsn";

    private const string ConnectionsSql = @"
        SELECT client_addr::text AS ClientAddr,
               application_name AS ApplicationName,
               state AS State,
               sent_lsn::text AS SentLsn,
               replay_lsn::text AS ReplayLsn,
               EXTRACT(EPOCH FROM write_lag)::float8 AS WriteLagSeconds,
               EXTRACT(EPOCH FROM flush_lag)::float8 AS FlushLagSeconds,
               EXTRACT(EPOCH FROM replay_lag)::float8 AS ReplayLagSeconds,
               pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)::bigint AS ReplayLagBytes,
               sync_state AS SyncState
        FROM pg_stat_replication
        ORDER BY application_name";

    private const string ReplicaSql = @"
        SELECT pg_is_in_recovery() AS InRecovery,
               pg_last_wal_receive_lsn()::text AS ReceiveLsn,
               pg_last_wal_replay_lsn()::text AS ReplayLsn,
               pg_last_xact_replay_timestamp() AS LastReplayAt,
               EXTRACT(EPOCH FROM now() - pg_last_xact_replay_timestamp())::float8 AS
LagSeconds";

    private readonly string _primaryConnectionString;
    private readonly string? _replicaConnectionString;
    private readonly ILogger<ReplicationRepository> _logger;

    /// <summary>
    /// Initializes a new instance of the <see cref="ReplicationRepository"/> class.
    /// </summary>
    /// <param name="primaryConnectionString">The connection string of the primary.</param>
    /// <param name="replicaConnectionString">The connection string of the replica; null when
there is none.</param>
    /// <param name="logger">The logger.</param>
    public ReplicationRepository(
        string primaryConnectionString, string? replicaConnectionString,
        ILogger<ReplicationRepository> logger)
    {
        _primaryConnectionString = primaryConnectionString;
        _replicaConnectionString = replicaConnectionString;
        _logger = logger;
    }
}
```

```

    /// <inheritdoc/>
    public async Task<ReplicationStatus> GetStatusAsync(Cancellation_token cancellation_token =
    default)
    {
        var status = new ReplicationStatus { ReplicaConfigured =
        !string.IsNullOrEmpty(_replicaConnectionString) };

        try
        {
            await using var primary = new NpgsqlConnection(_primaryConnectionString);
            await primary.OpenAsync(cancellation_token);

            var primaryRow = await primary.QuerySingleAsync<PrimaryRow>(PrimarySql);
            status.PrimaryLsn = primaryRow.Lsn;
            status.Connections = (await
            primary.QueryAsync<ReplicaConnection>(ConnectionsSql)).ToList();

            if (!status.ReplicaConfigured)
            {
                return status;
            }

            await using var replica = new NpgsqlConnection(_replicaConnectionString);
            await replica.OpenAsync(cancellation_token);

            var replicaRow = await replica.QuerySingleAsync<ReplicaRow>(ReplicaSql);
            status.ReplicaInRecovery = replicaRow.InRecovery;
            status.ReplicaReceiveLsn = replicaRow.ReceiveLsn;
            status.ReplicaReplayLsn = replicaRow.ReplayLsn;
            status.LastReplayAt = replicaRow.LastReplayAt;
            status.LagSeconds = replicaRow.LagSeconds;

            // The lag in bytes is computed on the primary: it knows both positions
            if (!string.IsNullOrEmpty(replicaRow.ReplayLsn))
            {
                status.LagBytes = await primary.ExecuteScalarAsync<long>(
                "SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), @Lsn::pg_lsn)::bigint",
                new { Lsn = replicaRow.ReplayLsn });
            }
        }
        catch (Exception ex) when (ex is not OperationCanceledException)
        {
            status.Error = ex.Message;
            _logger.LogError(ex, "Failed to read replication status");
        }

        return status;
    }

    private sealed class PrimaryRow
    {
        public bool InRecovery { get; set; }

        public string Lsn { get; set; } = string.Empty;
    }

    private sealed class ReplicaRow
    {
        public bool InRecovery { get; set; }

        public string? ReceiveLsn { get; set; }

        public string? ReplayLsn { get; set; }

        public DateTime? LastReplayAt { get; set; }

        public double? LagSeconds { get; set; }
    }

```

```
}  
}
```